

TP 8 - Scalabilité horizontale ascendante et descendante avec serveurs web, load balancing et base répliquée

1. Objectif du TP

À la fin de ce TP, vous devez être capables de :

- déployer une architecture web scalable sur KVM
- mettre en place une **scalabilité horizontale ascendante**
- mettre en place une **scalabilité horizontale descendante**
- relier tous les serveurs web à une **base de données principale**
- mettre en place une **réplication de base de données**
- isoler la réplication sur un **réseau séparé**
- distinguer les rôles de :
 - load balancer
 - serveurs web
 - base principale
 - base répliquée
 - réseaux de service et de réplication
- documenter les choix d'architecture, les flux et les limites

2. Idée centrale du TP

Dans ce TP, vous ne cherchez pas seulement à faire fonctionner un site web.

Vous devez construire une architecture qui sait :

- **monter en capacité** quand la charge augmente
- **réduire le nombre d'instances** quand la charge redescend
- **conserver une cohérence d'accès à la base**
- **répliquer la base sur un réseau isolé**
- **continuer à rendre le service sans casser l'architecture**

Formule à retenir

- **scalabilité ascendante horizontale** = vous ajoutez des serveurs web
 - **scalabilité descendante horizontale** = vous retirez des serveurs web quand ils ne sont plus nécessaires
-

3. Architecture cible attendue

Vous devez viser une architecture de ce type :

Couche d'accès

- 1 point d'entrée de service
- 1 load balancer

Couche web

- 2 serveurs web minimum au démarrage
- jusqu'à 3 ou 4 serveurs web si la charge augmente
- retour à 2 serveurs web quand la charge baisse

Couche base de données

- 1 base principale
- 1 base répliquée

Réseaux distincts

- 1 réseau frontal pour l'accès utilisateur
- 1 réseau de service entre web et base
- 1 réseau dédié à la réplication de base de données

Cette logique s'inscrit directement dans les séances sur réseau virtuel, segmentation, cluster/réplication, supervision et résilience.

6. Contraintes imposées

Vous devez respecter les contraintes suivantes :

Contraintes de capacité

- au démarrage, il doit y avoir **2 serveurs web**
- en cas de montée en charge, vous devez pouvoir passer à **3 puis 4 serveurs web**
- lorsque la charge baisse, vous devez pouvoir revenir progressivement à **2 serveurs web**

Contraintes de base de données

- tous les serveurs web doivent utiliser la **même base principale**
- la base principale doit être **répliquée** vers une deuxième base
- le trafic de réplication doit passer par un **réseau dédié**

Contraintes réseau

- vous devez séparer :
 - le trafic utilisateur
 - le trafic web vers base
 - le trafic de réplication

Contraintes pédagogiques

- vous devez justifier vos choix
 - vous ne devez pas seulement montrer “que ça marche”
 - vous devez démontrer que vous comprenez la logique de l’architecture
-

7. Travail demandé

Partie A - Concevoir le schéma d’architecture

Vous devez produire un schéma clair montrant :

- le load balancer
- les serveurs web
- la base principale
- la base répliquée
- les trois réseaux
- les flux autorisés

Votre schéma doit faire apparaître

- le réseau frontal
- le réseau applicatif
- le réseau de réplication
- le sens des flux
- les zones qui ne doivent pas communiquer directement

Vous devez expliquer

- pourquoi la réplication ne doit pas passer par le même réseau que le trafic utilisateur
 - pourquoi le load balancer ne doit pas parler directement à la base
 - pourquoi les serveurs web doivent être interchangeables
-

Partie B - Préparer une image modèle de serveur web

Vous devez préparer une image modèle qui servira à créer vos serveurs web.

Travail demandé

Vous devez :

- définir une VM de référence
- y installer le service web nécessaire
- la nettoyer et la préparer au clonage
- prévoir sa personnalisation au premier démarrage
- documenter la logique retenue

Objectif

Vous devez être capables de créer rapidement :

- web01
- web02
- web03
- web04

Question

Pourquoi une image modèle est-elle indispensable dans une logique de scalabilité horizontale ?

La progression prévoit explicitement l'industrialisation des déploiements avec **templates**, **cloud-init** et **scripts simples**.

Partie C - Déployer les 2 serveurs web initiaux

Vous devez démarrer avec **2 serveurs web actifs**.

Travail demandé

Chaque serveur web doit :

- exposer le même service
- afficher son identité
- être joignable par le load balancer
- accéder à la base principale
- être remplaçable par un clone identique

Attendu

Vous devez démontrer que :

- les deux serveurs rendent le même service
- ils utilisent la même base logique
- ils sont interchangeables du point de vue du client

Partie D - Déployer la base principale et la base répliquée

Vous devez mettre en place :

- **db01** : base principale
- **db02** : base répliquée

Travail demandé

Vous devez :

- choisir un moteur de base de données
- justifier le mode de réplication retenu
- documenter le sens de réplication
- définir le rôle de chaque serveur

Point attendu

Vous devez être capables d'expliquer :

- pourquoi les serveurs web écrivent sur la base principale
 - à quoi sert la base répliquée
 - pourquoi la base répliquée n'est pas simplement "une copie de secours passive" sans logique de service
-

Partie E - Séparer les réseaux

Vous devez construire au minimum **3 réseaux logiques** :

Réseau 1 - Réseau frontal

Utilisé pour :

- les clients
- le point d'entrée du service
- le trafic vers le load balancer

Réseau 2 - Réseau applicatif

Utilisé pour :

- les flux entre les serveurs web et la base principale

Réseau 3 - Réseau de réplication

Utilisé uniquement pour :

- les échanges entre la base principale et la base répliquée

Travail demandé

Vous devez justifier :

- pourquoi cette séparation améliore la sécurité
- pourquoi cette séparation améliore la lisibilité
- pourquoi cette séparation peut aussi améliorer la performance

La progression insiste explicitement sur les **bridges, VLAN, segmentation, micro-segmentation logique et analyse de performance**.

Partie F - Mettre en place le load balancing

Vous devez installer un répartiteur de charge open source.

Travail demandé

Le load balancer doit :

- exposer un point d'entrée unique
- répartir le trafic entre les serveurs web
- vérifier la santé des backends
- retirer un serveur web de la rotation s'il est indisponible

Vous devez justifier

- l'algorithme de répartition retenu
 - la méthode de vérification de santé
 - le comportement attendu en cas de panne d'un serveur web
-

Partie G - Définir la scalabilité ascendante

Vous devez maintenant définir comment l'architecture **monte en charge**.

Travail demandé

Vous devez fixer des règles claires :

État initial

- 2 serveurs web actifs

Premier palier

- si la charge dépasse un seuil défini, vous ajoutez **web03**

Deuxième palier

- si la charge continue d'augmenter, vous ajoutez **web04**

Vous devez choisir et justifier :

- les indicateurs observés
- les seuils retenus
- le délai d'observation
- la manière de déployer une nouvelle instance
- les conditions d'intégration au load balancer

Indicateurs possibles

- CPU moyen
- temps de réponse
- nombre de connexions
- saturation observée
- métriques applicatives

La progression demande justement de **mettre en place des indicateurs de performance pour anticiper les dérives**.

Partie H - Définir la scalabilité descendante

Vous devez maintenant prévoir la **réduction de capacité** quand la charge diminue.

Travail demandé

Vous devez définir comment :

- passer de 4 à 3 serveurs web
- puis de 3 à 2 serveurs web
- sans interrompre le service
- sans supprimer brutalement une instance encore en cours d'utilisation

Vous devez expliquer

- quels critères permettent de retirer une instance
- comment la sortir proprement de la rotation
- à quel moment vous la stoppez réellement
- pourquoi il ne faut jamais réduire trop vite

Point attendu

Vous devez montrer que la scalabilité descendante est plus délicate qu'elle n'en a l'air :

- retirer une VM trop tôt peut casser des sessions
 - retirer une VM sans méthode peut dégrader le service
 - un scale-in mal conçu est aussi dangereux qu'un scale-out mal conçu
-

Partie I - Tester la montée en charge

Vous devez simuler une charge croissante.

Travail demandé

Vous devez :

- mesurer le comportement avec 2 serveurs web
- provoquer une augmentation de charge
- justifier l'ajout de web03
- poursuivre si besoin jusqu'à web04
- vérifier que le load balancer répartit bien les requêtes

Ce que vous devez prouver

- que la plateforme sait accueillir de nouvelles instances
 - que ces nouvelles instances sont correctement intégrées
 - que la base principale reste le point d'accès unique pour les données
 - que la réplication continue de fonctionner
-

Partie J - Tester la descente en charge

Vous devez ensuite simuler un retour à la normale.

Travail demandé

Vous devez :

- faire diminuer la charge
- sortir progressivement une instance de la rotation
- vérifier que le service reste stable
- revenir à 2 serveurs web actifs
- documenter chaque étape

Question

Pourquoi ne doit-on pas supprimer une instance web “d’un coup” sans précaution ?

Partie K - Vérifier la cohérence côté base de données

La scalabilité web ne doit pas casser la cohérence des données.

Travail demandé

Vous devez démontrer que :

- les nouveaux serveurs web utilisent bien la même base principale
- les opérations réalisées par un serveur web sont visibles par les autres
- la base répliquée reçoit bien les changements
- le réseau de réplication est bien distinct du réseau de service

Vous devez expliquer

- ce que devient la base répliquée dans votre architecture
 - si elle sert uniquement de reprise
 - ou si elle pourrait servir à d’autres usages selon votre design
-

Partie L - Tester un incident

Vous devez maintenant provoquer un incident contrôlé.

Choix possibles

- panne d’un serveur web
- panne du load balancer
- indisponibilité temporaire de la base répliquée

- saturation du lien de réplication
- erreur de segmentation réseau

Travail demandé

Vous devez analyser :

- l'impact sur le service
- l'impact sur la capacité
- l'impact sur les données
- l'impact sur la réplication

Point attendu

Vous devez distinguer :

- panne de capacité
- panne de disponibilité
- panne de cohérence
- panne réseau

Partie M - Analyse finale

Vous devez conclure sur les notions suivantes :

Vous devez expliquer clairement

- ce qu'est la **scalabilité horizontale ascendante**
- ce qu'est la **scalabilité horizontale descendante**
- pourquoi l'image modèle facilite la montée en charge
- pourquoi le load balancer est indispensable

- pourquoi la base doit être pensée différemment des serveurs web
- pourquoi la réplication doit être isolée sur un autre réseau
- quelles limites subsistent malgré tout